

Bloque 9 — Mini-especificaciones funcionales y técnicas

Banco y conciliación: B010 a B017

1. Propósito del bloque

Este bloque implementa la conciliación bancaria avanzada y el control formal de períodos.

Incluye:

- Distribución de un movimiento bancario entre varias facturas.
- Aplicaciones parciales de movimientos y operaciones.
- Cálculo de saldos pendientes.
- Reglas configurables de cruce automático.
- Sugerencias de coincidencias.
- Cruce manual asistido.
- Integración bancaria con facturación, rendiciones y factoring.
- Exclusión justificada de movimientos.
- Reversión de aplicaciones.
- Reapertura de conciliaciones cerradas.
- Cierre de períodos por cuenta bancaria.
- Conservación de snapshots y evidencia de cierre.

Al finalizar este bloque, el sistema podrá responder:

¿Qué operación explica cada movimiento bancario?
¿Cuánto del movimiento está aplicado?
¿Qué saldo continúa pendiente?
¿Qué facturas fueron pagadas con un depósito?
¿Qué fondos, devoluciones o reembolsos corresponden a un egreso o ingreso?
¿Qué movimientos fueron excluidos y por qué?
¿Quién revirtió una conciliación?
¿El período bancario puede cerrarse?

2. Resumen del bloque

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
B010	Un movimiento paga varias facturas	12 h	Integraciones	Parcial	Distribución bancaria y documental
B011	Conciliación parcial	10 h	Backend	Alta, con ajustes	SalDOS y estados calculados
B012	Reglas de cruce automático	12 h	Backend	No cubierta	Motor de reglas y sugerencias
B013	Cruce manual asistido	8 h	Frontend	Parcial	Selección guiada de candidatos
B014	Cruce con rendiciones y facturas	12 h	Integraciones	Alta, con ampliaciones	Puente transversal completo
B015	Exclusiones justificadas	4 h	Backend	Parcial	Exclusión sin alterar cartola
B016	Reversas y reapertura	8 h	Backend	Parcial	Deshacer con permisos e historial
B017	Cierre de período	10 h	Backend	Parcial	Bloqueo por cuenta y período

El bloque suma **76 horas**.

La Carta Gantt define:

- B010: distribución por monto sin exceder saldos ni movimiento.
- B011: saldo aplicado, pendiente y estado calculado.
- B012: coincidencia por monto, referencia, RUT y fecha.
- B013: búsqueda y selección de operaciones candidatas.
- B014: enlace con pagos, devoluciones, reembolsos y factoring.
- B015: excluir sin perder auditoría ni modificar la cartola.
- B016: deshacer conciliación con motivo y permisos.
- B017: bloqueo por cuenta y período con diferencias controladas.

3. Orden técnico recomendado

La numeración de la Gantt se conserva, pero la construcción interna recomendada es:

B011 Conciliación parcial

- B010 Un movimiento paga varias facturas
- B013 Cruce manual asistido
- B014 Cruce transversal
- B012 Reglas automáticas
- B015 Exclusiones
- B016 Reversas y reapertura
- B017 Cierre de período

Razón del orden

B011 define el motor común de:

- Monto aplicado.
- Saldo disponible.
- Estado parcial.
- Límites de origen.
- Límites de destino.

B010, B013 y B014 reutilizan ese motor.

B012 debe implementarse después de que los cruces manuales estén estabilizados. Las reglas automáticas deben representar casos ya comprendidos y probados, no intentar definir el proceso antes que los usuarios sepan cómo conciliar correctamente.

4. Modelo conceptual de conciliación

La conciliación se basa en tres niveles.

4.1 Movimiento bancario

Representa lo informado por el banco:

MovimientoBancario

Puede ser:

INGRESO
EGRESO

El modelo actual ya calcula:

```
monto aplicado
saldo por conciliar
estado calculado
```

y distingue movimientos no conciliados, parciales, conciliados y excluidos.

4.2 Aplicación bancaria

Representa qué operación explica una parte del movimiento:

```
AplicacionMovimientoBancario
```

Destinos actuales:

```
PagoCliente
FlujoFactoring
EntregaFondo
PagoLiquidacionRendicion
DevolucionPagoCliente
Otro
```

El modelo ya exige un destino, controla que el movimiento no sea sobreaplicado y valida parcialmente la dirección ingreso/egreso.

4.3 Aplicación documental o funcional

Algunos destinos tienen una segunda distribución.

Ejemplo de factura:

```
MovimientoBancario
→ AplicacionMovimientoBancario
→ PagoCliente
→ AplicacionPagoFactura
→ FacturaVenta
```

Ejemplo de rendición:

```
MovimientoBancario
→ AplicacionMovimientoBancario
```

- EntregaFondo
- Rendicion

Ejemplo de regularización:

- MovimientoBancario
- AplicacionMovimientoBancario
- PagoLiquidacionRendicion
- LiquidacionRendicion

La conciliación bancaria no debe saltarse estos niveles creando relaciones directas inconsistentes.

5. Principios comunes del bloque

5.1 No modificar la cartola

Los siguientes campos representan información original y permanecen inalterables:

- fecha_operacion
- tipo
- monto
- descripcion_original
- referencia_bancaria
- identificador_externo
- contraparte
- fingerprint

Clasificar, aplicar, excluir o revertir no debe modificar el registro importado.

5.2 No borrar aplicaciones

Una aplicación financiera no se elimina físicamente.

El ciclo será:

- Aplicación activa
- reversada o anulada
- permanece en historial

5.3 Control de ambos lados

No basta con comprobar:

```
Suma aplicada ≤ movimiento
```

También debe comprobarse:

```
Suma recibida por el destino  
≤ monto disponible del destino
```

Ejemplo:

```
Movimiento:           1.000.000 CLP  
PagoCliente:          600.000 CLP  
  
No puede aplicarse:  
Movimiento → PagoCliente    800.000 CLP
```

Aunque el movimiento tenga saldo, el destino no puede recibir más de su propio monto.

5.4 Transacciones

Toda operación de conciliación debe utilizar:

```
transaction.atomic()
```

y bloquear los registros monetarios mediante:

```
select_for_update()
```

Como mínimo:

- Movimiento bancario.
 - Operación destino.
 - Aplicaciones activas.
 - Conciliación del período, cuando exista.
-

5.5 Idempotencia

Confirmar dos veces la misma operación no debe crear aplicaciones duplicadas.

Cada petición crítica puede utilizar:

```
clave de idempotencia
correlation_id
```

para detectar reintentos del navegador, API o usuario.

6. Modelos nuevos y ampliaciones comunes

6.1 Regla de cruce bancario

```
class ReglaCruceBancario(ModeloAuditoria):
    class Modo(models.TextChoices):
        SUGERIR = "SUGERIR", "Generar sugerencia"
        APLICAR = "APLICAR", "Aplicar automáticamente"

    class Destino(models.TextChoices):
        PAGO_CLIENTE = "PAGO_CLIENTE", "Pago de cliente"
        ENTREGA_FONDO = "ENTREGA_FONDO", "Entrega de fondo"
        PAGO_LIQUIDACION = (
            "PAGO_LIQUIDACION",
            "Pago de liquidación",
        )
        DEVOLUCION_PAGO = (
            "DEVOLUCION_PAGO",
            "Devolución a cliente",
        )
        FLUJO_FACTORING = (
            "FLUJO_FACTORING",
            "Flujo de factoring",
        )

    nombre = models.CharField(max_length=150)

    cuenta_bancaria = models.ForeignKey(
        CuentaBancaria,
        on_delete=models.PROTECT,
        null=True,
        blank=True,
        related_name="reglas_cruce",
    )

    tipo_movimiento = models.CharField(
        max_length=10,
        choices=MovimientoBancario.Tipo.choices,
        blank=True,
    )
```

```

destino = models.CharField(
    max_length=30,
    choices=Destino.choices,
)

modo = models.CharField(
    max_length=20,
    choices=Modo.choices,
    default=Modo.SUGERIR,
)

prioridad = models.PositiveIntegerField(default=100)
puntaje_minimo = models.PositiveIntegerField(default=80)

tolerancia_monto = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

tolerancia_dias = models.PositiveIntegerField(default=3)

exige_monto_exacto = models.BooleanField(default=False)
exige_referencia = models.BooleanField(default=False)
exige_rut = models.BooleanField(default=False)
exige_contraparte = models.BooleanField(default=False)

patron_descripcion = models.CharField(
    max_length=255,
    blank=True,
)

activa = models.BooleanField(default=True)
version = models.PositiveIntegerField(default=1)
observaciones = models.TextField(blank=True)

```

6.2 Sugerencia de cruce

```

class SugerenciaCruceBancario(ModeloAuditoria):
    class Estado(models.TextChoices):
        PENDIENTE = "PENDIENTE", "Pendiente"
        ACEPTADA = "ACEPTADA", "Aceptada"
        RECHAZADA = "RECHAZADA", "Rechazada"
        EXPIRADA = "EXPIRADA", "Expirada"
        APLICADA = "APLICADA", "Aplicada"

    movimiento = models.ForeignKey(

```

```

        MovimientoBancario,
        on_delete=models.CASCADE,
        related_name="sugerencias_cruce",
    )

    regla = models.ForeignKey(
        ReglaCruceBancario,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="sugerencias",
    )

    destino = models.CharField(max_length=30)
    objeto_destino_id = models.PositiveBigIntegerField()
    puntaje = models.PositiveIntegerField()
    monto_sugerido = models.DecimalField(
        max_digits=14,
        decimal_places=2,
    )

    razones = models.JSONField(default=dict)
    estado = models.CharField(
        max_length=20,
        choices=Estado.choices,
        default=Estado.PENDIENTE,
    )

    revisada_por = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="sugerencias_cruce_revisadas",
    )

    revisada_en = models.DateTimeField(
        null=True,
        blank=True,
    )

    motivo_rechazo = models.TextField(blank=True)

```

Para evitar una relación genérica insegura, la implementación final puede utilizar FKs opcionales similares a `AplicacionMovimientoBancario`.

6.3 Reversa de aplicación

Se recomienda ampliar `AplicacionMovimientoBancario`:

```

anulada_en = models.DateTimeField(
    null=True,
    blank=True,
)

anulada_por = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name="aplicaciones_bancarias_anuladas",
)

motivo_anulacion = models.TextField(blank=True)

correlation_id = models.UUIDField(
    default=uuid.uuid4,
    editable=False,
)

```

El campo `activa` se mantiene para compatibilidad y consultas rápidas.

6.4 Ampliación de exclusión

El modelo actual `ExclusionMovimientoBancario` ya relaciona un movimiento, motivo, usuario y fecha, y exige una justificación no vacía.

Se recomienda agregar:

```

class Tipo(models.TextChoices):
    MOVIMIENTO_INTERNO = (
        "MOVIMIENTO_INTERNO",
        "Movimiento interno",
    )
    SALDO_INICIAL = "SALDO_INICIAL", "Saldo inicial"
    COMISION_BANCARIA = (
        "COMISION_BANCARIA",
        "Comisión bancaria",
    )
    INTERES_BANCARIO = (
        "INTERES_BANCARIO",
        "Interés bancario",
    )
    AJUSTE_BANCARIO = "AJUSTE_BANCARIO", "Ajuste bancario"
    SIN_OPERACION_INTERNA = (
        "SIN_OPERACION_INTERNA",
        "Sin operación interna asociada",
    )

```

```

    )
    OTRO = "OTRO", "Otro"

    tipo = models.CharField(
        max_length=40,
        choices=Tipo.choices,
        default=Tipo.OTRO,
    )

    activa = models.BooleanField(default=True)

    revocada_en = models.DateTimeField(
        null=True,
        blank=True,
    )

    revocada_por = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="exclusiones_bancarias_revocadas",
    )

    motivo_revocacion = models.TextField(blank=True)

```

6.5 Ampliación de Conciliación Bancaria

El modelo actual contiene cuenta, período, saldo libro, saldo banco, diferencia, estado y datos de cierre.

Se recomienda agregar:

```

class OrigenSaldoLibro(models.TextChoices):
    MANUAL = "MANUAL", "Informado manualmente"
    SISTEMA = "SISTEMA", "Calculado por el sistema"
    INTEGRACION = "INTEGRACION", "Importado desde contabilidad"

    origen_saldo_libro = models.CharField(
        max_length=20,
        choices=OrigenSaldoLibro.choices,
        default=OrigenSaldoLibro.MANUAL,
    )

    saldo_inicial_banco = models.DecimalField(
        max_digits=14,
        decimal_places=2,
        null=True,
        blank=True,
    )

```

```

)

saldo_final_banco = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    null=True,
    blank=True,
)

total_ingresos = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

total_egresos = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

total_aplicado = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

total_excluido = models.DecimalField(
    max_digits=14,
    decimal_places=2,
    default=Decimal("0.00"),
)

cantidad_pendientes = models.PositiveIntegerField(default=0)
cantidad_parciales = models.PositiveIntegerField(default=0)
cantidad_excluidos = models.PositiveIntegerField(default=0)

snapshot = models.JSONField(default=dict, blank=True)
motivo_cierre_con_diferencia = models.TextField(blank=True)

reabierta_en = models.DateTimeField(
    null=True,
    blank=True,
)

reabierta_por = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name="conciliaciones_reabiertas",
)

```

```
)  
  
motivo_reapertura = models.TextField(blank=True)  
numero_revision = models.PositiveIntegerField(default=1)
```

B010 — Un movimiento paga varias facturas

1. Objetivo

Permitir que un depósito bancario sea distribuido entre una o varias facturas, sin crear una relación directa que omita el registro del pago del cliente.

Ejemplo:

Movimiento bancario:	1.000.000 CLP
Factura 1001:	400.000 CLP
Factura 1002:	350.000 CLP
Factura 1003:	250.000 CLP

Resultado:

Movimiento
→ PagoCliente por 1.000.000 CLP
→ tres AplicacionPagoFactura

2. Flujo correcto

MovimientoBancario
→ AplicacionMovimientoBancario
→ PagoCliente
→ AplicacionPagoFactura
→ FacturaVenta

No se recomienda crear:

MovimientoBancario → FacturaVenta

porque se perderían:

- Receptor del pago.

- Medio de pago.
- Estado del pago.
- Excedentes.
- Distribución.
- Casos de factoring.
- Historial de aplicación.

3. Casos soportados

Un movimiento, un cliente, varias facturas

Se crea o selecciona un `PagoCliente`.

Después se distribuye entre las facturas.

Un movimiento, varios clientes

Ejemplo de depósito agrupado:

Movimiento:	1.500.000 CLP
Cliente A:	900.000 CLP
Cliente B:	600.000 CLP

Se deben crear dos pagos:

<code>PagoCliente A</code>
<code>PagoCliente B</code>

y dos aplicaciones bancarias.

Movimiento mayor que las facturas

El excedente permanece como:

<code>saldo sin aplicar del PagoCliente</code>
--

o:

<code>saldo por conciliar del movimiento</code>

según la decisión del usuario.

4. Asistente funcional

Paso 1 — Revisar movimiento

Mostrar:

- Cuenta.
- Fecha.
- Monto.
- Descripción.
- Referencia.
- Contraparte.
- Saldo disponible.

Paso 2 — Seleccionar cliente

Buscar por:

- Nombre.
- RUT.
- Referencia.
- Contraparte.

Paso 3 — Seleccionar facturas

Mostrar solo:

- Facturas del cliente.
- No anuladas.
- Con saldo.
- Compatibles con factoring.

Paso 4 — Distribuir monto

Tabla:

Factura	Vencimiento	Saldo	Aplicar
1001	10-09-2026	400.000 CLP	400.000 CLP
1002	15-09-2026	350.000 CLP	350.000 CLP
1003	20-09-2026	500.000 CLP	250.000 CLP

Paso 5 — Confirmar

La operación crea en una sola transacción:

- PagoCliente.
- AplicacionMovimientoBancario.
- AplicacionPagoFactura.
- Estados.

- Auditoría.
-

5. Pago existente o nuevo

El usuario podrá:

Usar pago existente

Cuando el pago ya fue registrado por cobranza.

Crear pago desde el movimiento

El sistema propone:

- Cliente.
- Fecha.
- Monto.
- Referencia.
- Medio de pago.
- Receptor.

La creación sigue siendo explícita y auditable.

6. Reglas

1. El movimiento debe ser de ingreso.
 2. El movimiento no puede estar excluido.
 3. Debe tener saldo disponible.
 4. Las facturas deben pertenecer al cliente del pago.
 5. Ninguna factura puede estar anulada.
 6. No se supera el movimiento.
 7. No se supera el pago.
 8. No se supera ninguna factura.
 9. Las facturas cedidas se tratan según factoring.
 10. El pago puede quedar parcialmente distribuido.
 11. El movimiento puede quedar parcialmente conciliado.
 12. Toda la operación es atómica.
 13. Un error en una factura revierte la operación completa.
 14. El backend recalcula todos los montos.
 15. No se confía en los totales JavaScript.
-

7. Servicio

`services/conciliacion/pagos_facturas.py`

```

@transaction.atomic
def aplicar_movimiento_a_facturas(
    *,
    movimiento_id,
    cliente_id,
    aplicaciones_factura,
    usuario,
    pago_id=None,
    datos_pago=None,
):
    movimiento = (
        MovimientoBancario.objects
        .select_for_update()
        .get(pk=movimiento_id)
    )

    # Obtener o crear pago.
    # Bloquear facturas.
    # Validar saldos.
    # Crear aplicación bancaria.
    # Crear aplicaciones a facturas.
    # Actualizar estados.
    # Registrar correlation_id.

```

8. URLs

```

/administracion/movimientos/<pk>/pagar-facturas/
/administracion/movimientos/<pk>/pagar-facturas/candidatos/
/administracion/movimientos/<pk>/pagar-facturas/confirmar/

```

9. Vistas

```

MovimientoPagarFacturasView
MovimientoFacturasCandidatasView
MovimientoConfirmarPagoFacturasView

```

10. Permisos

```
aplicar_movimiento_a_facturas  
crear_pago_desde_movimiento  
aplicar_pago_a_facturas
```

11. Auditoría

Registrar:

- Movimiento.
 - Pago creado o utilizado.
 - Cliente.
 - Facturas.
 - Montos.
 - Saldos anteriores y nuevos.
 - Estados.
 - Usuario.
 - Identificador de operación.
-

12. Pruebas

- Una factura.
 - Varias facturas.
 - Aplicación parcial.
 - Excedente.
 - Pago existente.
 - Pago nuevo.
 - Varios clientes.
 - Factura de otro cliente.
 - Factura anulada.
 - Factura cedida.
 - Exceso sobre movimiento.
 - Exceso sobre factura.
 - Error intermedio y rollback.
 - Confirmación doble.
 - Concurrencia.
-

13. Criterios de aceptación

- Un movimiento paga varias facturas.
- La relación pasa por `PagoCliente`.
- No se exceden los saldos.
- Se soportan pagos existentes y nuevos.

- Los estados se recalculan.
- La operación es atómica.
- Existe trazabilidad completa.

B011 — Conciliación parcial

1. Objetivo

Permitir que tanto el movimiento como la operación destino sean conciliados en varias etapas.

La Gantt exige mostrar saldo aplicado, saldo pendiente y estado calculado.

2. Conciliación parcial del movimiento

Movimiento:	1.000.000 CLP
Aplicado:	600.000 CLP
Pendiente:	400.000 CLP
Estado:	PARCIAL

3. Conciliación parcial del destino

PagoCliente:	1.000.000 CLP
Movimiento A:	600.000 CLP
Pendiente bancario:	400.000 CLP

Ambas perspectivas deben ser visibles.

4. Estados del movimiento

El modelo ya contempla:

NO_CONCILIADO
PARCIAL
CONCILIADO
EXCLUIDO

y calcula el estado a partir de las aplicaciones activas.

5. Estados de destinos

Se deben exponer propiedades equivalentes para:

```
PagoCliente  
FlujoFactoring  
EntregaFondo  
PagoLiquidacionRendicion  
DevolucionPagoCliente
```

Ejemplo genérico:

```
@property  
def monto_conciliado_banco(self):  
    ...  
  
@property  
def saldo_por_conciliar_banco(self):  
    return self.monto - self.monto_conciliado_banco
```

6. Estados sugeridos

```
NO_CONCILIADO  
PARCIAL  
CONCILIADO  
NO_REQUIERE_CONCILIACION  
ANULADO
```

No es obligatorio persistirlos si pueden anotarse eficientemente.

7. Reglas

1. Una aplicación parcial es válida.
2. El saldo no puede ser negativo.
3. Una aplicación anulada no suma.
4. Una aplicación pendiente no debe presentarse como conciliada.
5. Los movimientos excluidos no aceptan aplicaciones.
6. Los destinos anulados no aceptan aplicaciones.
7. La moneda debe coincidir.
8. El estado se actualiza después de cada cambio.
9. Los listados deben usar anotaciones.
10. Los cálculos utilizan `Decimal`.
11. Un período cerrado bloquea nuevas aplicaciones.
12. Las diferencias deben ser visibles.

8. Servicio común

```
services/conciliacion/saldos.py
```

Funciones:

```
calcular_saldo_movimiento()  
calcular_saldo_destino()  
actualizar_estado_movimiento()  
obtener_estado_destino()  
validar_monto_aplicable()  
validar_saldos_aplicacion()
```

9. Interfaz

El componente reutilizable mostrará:

```
Monto  
Aplicado  
Disponible  
Estado
```

Puede incorporarse en:

- Lista de movimientos.
- Detalle.
- Pagos.
- Fondos.
- Liquidaciones.
- Factoring.

10. Pruebas

- Cero aplicaciones.
- Una parcial.
- Varias parciales.
- Aplicación completa.
- Anulación.
- Recuperación de saldo.
- Exceso.
- Precisión decimal.
- Destino parcialmente conciliado.

- Listados anotados.
 - Período cerrado.
-

11. Criterios de aceptación

- El movimiento puede quedar parcial.
 - El destino puede quedar parcial.
 - Los saldos son correctos.
 - Las aplicaciones anuladas no suman.
 - Los estados se actualizan.
 - No existen saldos negativos.
 - La interfaz muestra ambas perspectivas.
-

B012 — Reglas de cruce automático

1. Objetivo

Detectar coincidencias entre movimientos y operaciones utilizando reglas configurables.

Los criterios de la Gantt son:

Monto
Referencia
RUT
Fecha

2. Principio de seguridad

La primera versión debe funcionar principalmente como:

Motor automático de sugerencias

no como conciliación autónoma sin revisión.

El modo automático podrá habilitarse únicamente para reglas:

- De alta confianza.
 - Probadas.
 - Versionadas.
 - Autorizadas.
 - Con rollback.
-

3. Criterios de coincidencia

Monto

- Exacto.
- Dentro de tolerancia.
- Suma de operaciones igual al movimiento.

Fecha

- Mismo día.
- Rango configurable.
- Fecha de pago versus fecha bancaria.

Referencia

- Coincidencia exacta.
- Coincidencia normalizada.
- Contiene número de factura.
- Contiene número de rendición.

RUT

- Contraparte bancaria.
- Cliente.
- Responsable.
- Empresa factoring.

Nombre

- Comparación normalizada de contraparte.
- Uso como señal secundaria.

Dirección

- Ingreso o egreso compatible.

Cuenta

- Regla general.
- Regla específica por cuenta.

4. Puntuación sugerida

Criterio	Puntaje
Monto exacto	40

Criterio	Puntaje
Referencia exacta	25
RUT exacto	20
Fecha dentro de tolerancia	10
Contraparte semejante	5

Ejemplo:

Puntaje 95
→ candidato fuerte

Los valores deben ser configurables o estar centralizados.

5. Umbrales

0-49: no sugerir
50-69: coincidencia débil
70-89: sugerencia revisable
90-99: alta confianza
100: coincidencia exacta

Solo una regla explícitamente autorizada podría aplicar automáticamente resultados sobre el umbral establecido.

6. Tipos de regla

- Movimiento a pago de cliente.
- Movimiento a entrega de fondo.
- Movimiento a devolución de responsable.
- Movimiento a reembolso.
- Movimiento a devolución de cliente.
- Movimiento a flujo de factoring.
- Movimiento a varias operaciones.

7. Versionado

Una regla utilizada no debe modificarse destruyendo su configuración histórica.

```
Regla versión 1 → inactiva
Regla versión 2 → activa
```

Cada sugerencia debe guardar:

- Regla.
- Versión.
- Puntaje.
- Razones.
- Fecha.

8. Ejecución

Comando:

```
python manage.py generar_sugerencias_conciliacion
```

Puede aceptar:

```
--cuenta
--desde
--hasta
--movimiento
```

Flujo:

```
Buscar movimientos pendientes
→ seleccionar reglas
→ obtener operaciones candidatas
→ puntuar
→ descartar incompatibles
→ guardar sugerencias
→ expirar sugerencias obsoletas
```

9. Aplicación automática

Cuando una regla use `modo=APLICAR` :

1. Debe existir un único candidato.
2. Debe superar el puntaje.
3. Los montos deben ser compatibles.
4. No debe haber conflicto con otra regla.

5. El período debe estar abierto.
 6. La operación debe continuar disponible.
 7. El servicio debe volver a validar todo.
 8. Debe registrarse como aplicación automática.
 9. Debe ser reversible.
-

10. URLs

```
/administracion/conciliacion/reglas/  
/administracion/conciliacion/reglas/nueva/  
/administracion/conciliacion/reglas/<pk>/  
/administracion/conciliacion/reglas/<pk>/probar/  
/administracion/conciliacion/reglas/<pk>/versionar/  
/administracion/conciliacion/sugerencias/
```

11. Vistas

```
ReglaCruceListView  
ReglaCruceCreateView  
ReglaCruceDetailView  
ReglaCruceProbarView  
ReglaCruceVersionarView  
SugerenciaCruceListView
```

12. Servicio

```
services/conciliacion/reglas.py  
services/conciliacion/sugerencias.py
```

Funciones:

```
evaluar_regla()  
calcular_puntaje()  
buscar_candidatos()  
generar_sugerencias()  
aceptar_sugerencia()  
rechazar_sugerencia()  
aplicar_sugerencia_automatica()  
expirar_sugerencias()
```

13. Auditoría

Registrar:

- Regla.
 - Versión.
 - Movimiento.
 - Candidato.
 - Puntaje.
 - Razones.
 - Aceptación.
 - Rechazo.
 - Aplicación automática.
 - Reversa.
-

14. Pruebas

- Monto exacto.
 - Referencia.
 - RUT.
 - Fecha.
 - Candidato único.
 - Varios candidatos.
 - Regla inactiva.
 - Regla de otra cuenta.
 - Dirección incompatible.
 - Puntaje bajo.
 - Puntaje alto.
 - Aplicación automática.
 - Conflicto entre reglas.
 - Ejecución repetida.
 - Regla versionada.
 - Sugerencia obsoleta.
-

15. Criterios de aceptación

- Las reglas son configurables.
 - Se generan sugerencias explicables.
 - El usuario conoce las razones del puntaje.
 - No se aplican candidatos incompatibles.
 - Las reglas quedan versionadas.
 - La ejecución es idempotente.
 - Las aplicaciones automáticas son reversibles.
-

B013 — Cruce manual asistido

1. Objetivo

Permitir al usuario buscar, comparar y seleccionar operaciones candidatas sin depender exclusivamente de reglas automáticas.

La Gantt solicita búsqueda y selección de operaciones candidatas.

2. Bandeja de trabajo

URL:

```
/administracion/conciliacion/asistida/
```

La pantalla puede trabajar en dos direcciones:

```
Movimiento → buscar operaciones  
Operación → buscar movimientos
```

3. Panel de movimientos

Filtros:

- Cuenta.
 - Fecha.
 - Tipo.
 - Monto.
 - Estado.
 - Clasificación.
 - Referencia.
 - Contraparte.
 - Con sugerencias.
 - Sin sugerencias.
 - Días pendiente.
-

4. Panel de candidatos

Por cada movimiento mostrar candidatos de:

- Pagos de clientes.
- Entregas de fondos.

- Liquidaciones.
- Devoluciones.
- Factoring.
- Otro destino autorizado.

5. Comparación

Criterio	Movimiento	Candidato	Coincidencia
Monto	500.000 CLP	500.000 CLP	Exacta
Fecha	15-09-2026	14-09-2026	1 día
RUT	12.345.678-9	12.345.678-9	Exacta
Referencia	FAC-1002	FAC-1002	Exacta
Contraparte	Cliente A	Cliente A	Alta

6. Acciones

- Aplicar monto completo.
- Aplicar parcial.
- Seleccionar varios destinos.
- Crear pago desde movimiento.
- Crear glosa.
- Guardar como sugerencia.
- Excluir.
- Dejar pendiente.
- Abrir operación candidata.

7. Selección múltiple

El usuario debe poder construir:

Movimiento

- └ destino A
- └ destino B
- └ destino C

La pantalla mostrará en tiempo real:

```
Monto del movimiento
Total seleccionado
Saldo restante
```

8. Validación final

El frontend puede alertar, pero el backend debe:

- Bloquear registros.
- Verificar saldos.
- Verificar direcciones.
- Verificar estados.
- Verificar período.
- Recalcular montos.
- Confirmar en una transacción.

9. Servicio

```
services/conciliacion/asistente.py
```

Funciones:

```
buscar_operaciones_candidatas()
comparar_movimiento_operacion()
construir_resumen_cruce()
confirmar_cruce_asistido()
guardar_decision_usuario()
```

10. URLs adicionales

```
/administracion/conciliacion/asistida/candidatos/
/administracion/conciliacion/asistida/comparar/
/administracion/conciliacion/asistida/confirmar/
```

11. Vistas

```
ConciliacionAsistidaView
ConciliacionCandidatosView
```

```
ConciliacionCompararView  
ConciliacionConfirmarView
```

12. Permisos

```
usar_conciliacion_asistida  
confirmar_conciliacion_asistida  
ver_candidatos_sensibles
```

13. Auditoría

Registrar:

- Candidatos presentados.
- Candidato seleccionado.
- Puntaje.
- Montos.
- Decisión.
- Usuario.
- Fecha.
- Descartes.

No es necesario conservar cada cambio visual, pero sí el conjunto final y la sugerencia aceptada o rechazada.

14. Pruebas

- Buscar por monto.
 - Buscar por fecha.
 - Buscar por RUT.
 - Buscar por referencia.
 - Aplicar completo.
 - Aplicar parcial.
 - Varios destinos.
 - Destino incompatible.
 - Período cerrado.
 - Exceso.
 - Confirmación concurrente.
 - Mantener filtros.
 - Permisos.
-

15. Criterios de aceptación

- El usuario puede localizar candidatos.
- Puede comparar información.
- Puede distribuir montos.
- El saldo restante es visible.
- La confirmación se revalida en backend.
- Las decisiones quedan auditadas.
- El proceso no depende de reglas automáticas.

B014 — Cruce con rendiciones y facturas

1. Objetivo

Completar el puente transversal entre cartola, facturación, rendiciones, devoluciones y factoring.

La tarea debe vincular movimientos con:

- Pagos.
- Devoluciones.
- Reembolsos.
- Factoring.

2. Matriz de destinos

Operación	Dirección bancaria	Modelo destino
Pago de cliente	Ingreso	PagoCliente
Anticipo de factoring	Ingreso	FlujoFactoring
Comisión de factoring	Egreso	FlujoFactoring
Liquidación del factor	Ingreso	FlujoFactoring
Entrega de fondo	Egreso	EntregaFondo
Devolución del responsable	Ingreso	PagoLiquidacionRendicion
Reembolso al responsable	Egreso	PagoLiquidacionRendicion
Devolución a cliente	Egreso	DevolucionPagoCliente

3. Facturas

El movimiento no se aplica directamente a la factura:

Movimiento
→ PagoCliente
→ Factura

Esto permite:

- Varias facturas.
- Pagos parciales.
- Excedentes.
- Factoring.
- Anulaciones.

4. Entregas de fondo

Movimiento de egreso
→ EntregaFondo
→ Rendicion

Reglas:

- La entrega debe corresponder al mismo monto o saldo.
- Debe existir compatibilidad de fecha.
- Una entrega anulada no puede vincularse.
- Una entrega completamente conciliada no recibe más aplicaciones.

5. Devolución del responsable

Movimiento de ingreso
→ PagoLiquidacionRendicion
tipo DEVOLUCION_RESPONSABLE

6. Reembolso al responsable

Movimiento de egreso
→ PagoLiquidacionRendicion
tipo REEMBOLSO_EMPRESA

7. Factoring

Movimiento
→ FlujoFactoring

La dirección se determina por el flujo:

- Anticipo: ingreso.
- Liquidación: ingreso.
- Comisión: egreso.
- Cobro con recurso: egreso.
- Devolución: según naturaleza.

8. Devolución a cliente

Movimiento de egreso
→ DevolucionPagoCliente

Debe comprobarse que:

- La devolución está confirmada.
- El monto no fue aplicado anteriormente.
- No se supera el saldo de devolución.

9. Servicio transversal

services/conciliacion/aplicaciones.py

Funciones:

```
crear_aplicacion_bancaria()
crear_aplicaciones_multiples()
validar_direccion_destino()
obtener_saldo_destino()
bloquear_destino()
actualizar_estado_destino()
actualizar_estado_movimiento()
```

Cada módulo puede exponer una función especializada, pero todas deben utilizar el mismo núcleo.

10. Corrección del modelo actual

La validación actual trata:

- PagoCliente como ingreso.
- EntregaFondo como egreso.
- DevolucionPagoCliente como egreso.
- FlujoFactoring según dirección.

Pero debe incorporar explícitamente `PagoLiquidacionRendicion`:

```
if destino == PAGO_LIQUIDACION:  
    if pago_liquidacion.tipo == DEVOLUCION_RESPONSABLE:  
        requiere INGRESO  
  
    if pago_liquidacion.tipo == REEMBOLSO_EMPRESA:  
        requiere EGRESO
```

11. Restricción de destino

Debe agregarse una `CheckConstraint` que garantice:

```
Destino seleccionado  
=  
única FK informada
```

La validación Python sigue siendo necesaria para mensajes comprensibles.

12. Interfaz

El detalle del movimiento mostrará destinos agrupados:

```
Facturación  
Rendiciones  
Factoring  
Devoluciones  
Otros
```

Cada grupo tendrá:

- Operaciones candidatas.
- Saldos.
- Dirección.
- Estado.

- Aplicar.
-

13. Pruebas

- Pago cliente.
 - Varias facturas.
 - Entrega fondo.
 - Devolución responsable.
 - Reembolso.
 - Devolución cliente.
 - Anticipo factoring.
 - Comisión factoring.
 - Dirección incorrecta.
 - Operación anulada.
 - Destino excedido.
 - Movimiento excedido.
 - Varios tipos en un movimiento.
 - Rollback transversal.
-

14. Criterios de aceptación

- Todos los destinos operativos están cubiertos.
 - Las direcciones se validan.
 - Los saldos de origen y destino se protegen.
 - Facturas pasan por pagos.
 - Rendiciones pasan por fondos o liquidaciones.
 - Factoring respeta la dirección del flujo.
 - La operación queda auditada.
-

B015 — Exclusiones justificadas

1. Objetivo

Marcar movimientos que no necesitan una aplicación operativa, sin eliminarlos ni modificar la cartola.

El modelo actual ya dispone de una exclusión uno a uno con motivo obligatorio.

2. Ejemplos de exclusión

- Saldo inicial.
- Movimiento interno.
- Comisión bancaria.
- Interés.
- Ajuste bancario.

- Línea informativa de la cartola.
- Operación fuera del alcance del sistema.
- Movimiento autorizado sin documento interno.

Un error de importación duplicado no debe resolverse mediante exclusión; debe corregirse en la importación o reversarse.

3. Efecto de excluir

La exclusión:

- Conserva el movimiento.
 - Mantiene su monto.
 - Mantiene su efecto en el saldo bancario.
 - Impide aplicaciones.
 - Lo retira de la bandeja ordinaria de pendientes.
 - Lo presenta como justificado en el cierre.
 - Registra usuario, fecha y motivo.
-

4. Condiciones

Solo puede excluirse cuando:

1. No tiene aplicaciones activas.
 2. No pertenece a un período cerrado, salvo reapertura.
 3. El usuario tiene permiso.
 4. Se selecciona tipo.
 5. Se ingresa motivo.
 6. Se adjunta respaldo cuando la política lo exige.
-

5. Estado

Movimiento.estado_conciliacion = EXCLUIDO

El movimiento no se considera conciliado con una operación, sino:

Justificado sin aplicación

Esta diferencia debe ser visible en reportes.

6. Revocación

Una exclusión puede revocarse:

EXCLUIDO
→ NO_CONCILIADO

Solo cuando:

- El período está abierto o reabierto.
- Se indica motivo.
- No existe otra exclusión activa.

La exclusión anterior permanece en historial.

7. URLs

```
/administracion/movimientos/<pk>/excluir/  
/administracion/movimientos/<pk>/revocar-exclusion/  
/administracion/movimientos/<pk>/historial-exclusion/
```

8. Vistas

```
MovimientoExcluirView  
MovimientoRevocarExclusionView  
MovimientoHistorialExclusionView
```

9. Servicio

```
services/conciliacion/exclusiones.py
```

Funciones:

```
excluir_movimiento()  
revocar_exclusion()  
validar_exclusion()  
actualizar_estado_movimiento()
```

10. Permisos

`excluir_movimientobancario`
`revocar_exclusionmovimientobancario`

11. Auditoría

Registrar:

- Movimiento.
 - Tipo.
 - Motivo.
 - Usuario.
 - Fecha.
 - Respaldo.
 - Revocación.
 - Período afectado.
-

12. Pruebas

- Excluir pendiente.
 - Excluir con aplicaciones.
 - Motivo vacío.
 - Tipo.
 - Revocar.
 - Período cerrado.
 - Usuario sin permiso.
 - Estado del movimiento.
 - Reporte de cierre.
 - Historial.
-

13. Criterios de aceptación

- El movimiento no se elimina.
 - La cartola no se modifica.
 - La exclusión requiere justificación.
 - El movimiento deja de aceptar aplicaciones.
 - La exclusión aparece en el cierre.
 - La revocación conserva historial.
 - Los permisos se respetan.
-

B016 — Reversas y reapertura

1. Objetivo

Deshacer conciliaciones erróneas sin eliminar evidencias, y permitir reabrir períodos cerrados bajo control excepcional.

La Gantt exige motivo y permisos.

2. Tipos de reversa

Reversa de aplicación bancaria

Desactiva una `AplicacionMovimientoBancario`.

Reversa de conjunto

Deshace todas las aplicaciones generadas en una misma operación o `correlation_id`.

Reversa de cruce con facturas

Debe revertir coherentemente:

```
AplicacionMovimientoBancario
AplicacionPagoFactura, cuando fue creada en la misma operación
PagoCliente, cuando corresponde
Estados
```

No debe anular automáticamente un pago que tenga otras relaciones válidas.

Revocación de exclusión

Devuelve el movimiento a pendiente.

Reapertura de período

Cambia una conciliación cerrada a abierta bajo una nueva revisión.

3. Principio de reversión

La reversa no consiste en borrar registros.

```
Aplicación activa
→ activa=False
```

→ motivo
→ usuario
→ fecha
→ estados recalculados

4. Dependencias posteriores

Antes de revertir se debe revisar si la aplicación produjo:

- Factura pagada.
- Rendición cerrada.
- Liquidación conciliada.
- Operación factoring cerrada.
- Devolución confirmada.
- Período cerrado.

Cuando existan dependencias:

Reversa bloqueada
o
reapertura coordinada obligatoria

5. Reapertura de período

Condiciones:

1. Conciliación cerrada.
2. Usuario con permiso especial.
3. Motivo obligatorio.
4. No existe otra conciliación abierta superpuesta.
5. Se incrementa `numero_revision`.
6. Se conserva el snapshot anterior.
7. Se registra usuario y fecha.
8. Se desbloquean únicamente los registros afectados.

Estado:

CERRADA
→ ABIERTA

Puede agregarse un estado explícito:

REABIERTA

pero también puede mantenerse `ABIERTA` junto con `numero_revision > 1`.

6. Snapshot de reapertura

Antes de reabrir se conserva:

```
{
  "revision": 1,
  "cerrado_en": "...",
  "saldo_libro": "...",
  "saldo_banco": "...",
  "diferencia": "...",
  "pendientes": 0,
  "exclusiones": []
}
```

El nuevo cierre generará otro snapshot.

7. Servicio

`services/conciliacion/reversas.py`

```
@transaction.atomic
def reversar_aplicacion(
    *,
    aplicacion_id,
    usuario,
    motivo,
):
    aplicacion = (
        AplicacionMovimientoBancario.objects
        .select_for_update()
        .select_related("movimiento")
        .get(pk=aplicacion_id)
    )

    # Validar estado.
    # Revisar dependencias.
    # Desactivar.
    # Recalcular origen.
    # Recalcular destino.
    # Registrar evento.
```

Funciones:

```
reversar_aplicacion()  
reversar_operacion_completa()  
validar_dependencias_reversa()  
reabrir_conciliacion()  
recalcular_objetos_afectados()
```

8. Interfaz

Antes de confirmar mostrar:

Esta reversa afectará:

- Movimiento bancario: PARCIAL → NO_CONCILIADO
- PagoCliente: CONCILIADO → PARCIAL
- Factura 1001: PAGADA → PARCIAL
- Período septiembre 2026: requiere reapertura

El usuario debe comprender el impacto antes de ejecutar.

9. URLs

```
/administracion/aplicaciones-bancarias/<pk>/reversar/  
/administracion/conciliacion/operaciones/<uuid>/reversar/  
/administracion/conciliaciones/<pk>/reabrir/  
/administracion/conciliaciones/<pk>/revisiones/
```

10. Vistas

```
AplicacionBancariaReversarView  
OperacionConciliacionReversarView  
ConciliacionReabrirView  
ConciliacionRevisionesView
```

11. Permisos

```
reversar_aplicacionbancaria  
reversar_operacionconciliacion  
reabrir_conciliacionbancaria
```

La reapertura debe reservarse para un rol superior.

12. Auditoría

Registrar:

- Aplicación.
 - Operación.
 - Motivo.
 - Usuario.
 - Estados anteriores.
 - Estados nuevos.
 - Período.
 - Snapshot anterior.
 - Objetos afectados.
-

13. Pruebas

- Reversa simple.
 - Reversa parcial.
 - Reversa de operación completa.
 - Aplicación ya anulada.
 - Motivo vacío.
 - Factura afectada.
 - Rendición cerrada.
 - Período cerrado.
 - Reapertura.
 - Número de revisión.
 - Segunda reapertura.
 - Concurrencia.
 - Usuario sin permiso.
 - Rollback.
-

14. Criterios de aceptación

- Las conciliaciones pueden revertirse.
 - No se eliminan aplicaciones.
 - Se muestran impactos.
 - Los estados se recalculan.
 - Los períodos cerrados exigen reapertura.
 - Los snapshots se conservan.
 - Toda reversa requiere motivo.
 - La operación es transaccional.
-

B017 — Cierre de período

1. Objetivo

Cerrar formalmente la conciliación de una cuenta bancaria para un intervalo de fechas y bloquear cambios ordinarios posteriores.

La Gantt exige bloqueo por cuenta y período con diferencias controladas.

2. Modelo principal

ConciliacionBancaria

El modelo existente ya representa cuenta, período, saldos, diferencia y cierre.

3. Limitación del alcance contable

Esta fase no implementa contabilidad general ni libro mayor completo.

Por tanto, `saldo_libro` debe indicar claramente su origen:

- Informado por el sistema contable.
- Ingresado por un usuario autorizado.
- Calculado como saldo operativo de control.

No debe presentarse como saldo contable oficial si no existe una integración o libro contable que lo respalde.

4. Creación del período

Campos:

- Cuenta bancaria.
 - Período desde.
 - Período hasta.
 - Origen del saldo libro.
 - Saldo inicial.
 - Saldo final del banco.
 - Saldo libro.
 - Observaciones.
-

5. Restricción de períodos

No debe existir más de una conciliación activa superpuesta para la misma cuenta.

Ejemplo inválido:

Conciliación A:
01-09-2026 a 30-09-2026

Conciliación B:
15-09-2026 a 15-10-2026

Se permitirá una nueva revisión del mismo período únicamente mediante reapertura.

6. Movimientos incluidos

Por defecto se consideran los movimientos cuya:

fecha_operacion

está dentro del período.

Debe permitirse incorporar excepcionalmente movimientos por fecha contable o valor mediante una decisión explícita.

Para mayor trazabilidad puede crearse:

```
class MovimientoPeriodoConciliacion(ModeloAuditoria):
    conciliacion = models.ForeignKey(
        ConciliacionBancaria,
        on_delete=models.PROTECT,
        related_name="movimientos_periodo",
    )

    movimiento = models.ForeignKey(
        MovimientoBancario,
        on_delete=models.PROTECT,
        related_name="periodos_conciliacion",
    )

    incluido = models.BooleanField(default=True)
    motivo = models.TextField(blank=True)
```

Esto congela qué movimientos formaron parte del cierre.

7. Validaciones previas

Antes de cerrar:

1. La cuenta está activa o autorizada para cierre histórico.
2. El período es válido.
3. No existe superposición.
4. Todas las importaciones del período están procesadas.
5. No existen importaciones con errores pendientes.
6. Todos los movimientos están:
7. Conciliados.
8. Excluidos justificadamente.
9. O aceptados como diferencia autorizada.
10. No existen aplicaciones inválidas.
11. No existen saldos negativos.
12. Las exclusiones tienen motivo.
13. Las reversas están resueltas.
14. El saldo banco está informado.
15. El saldo libro está informado.
16. La diferencia está calculada.
17. El usuario tiene permiso.

8. Diferencia

```
diferencia
=
saldo banco
- saldo libro
```

La fórmula debe definirse una vez y mantenerse consistente.

Diferencia cero

Cierre normal.

Diferencia dentro de tolerancia

Puede cerrarse si:

- Existe una tolerancia configurada.
- El usuario posee permiso.
- Se ingresa motivo.
- La diferencia queda visible.

Diferencia superior a tolerancia

Cierre bloqueado.

9. Resumen de cierre

Indicador	Resultado
Movimientos	1.250
Ingresos	150.000.000 CLP
Egresos	142.500.000 CLP
Conciliados	1.230
Excluidos	20
Pendientes	0
Saldo banco	25.500.000 CLP
Saldo libro	25.500.000 CLP
Diferencia	0 CLP

10. Snapshot

Al cerrar se almacena:

```
{
  "cuenta": {},
  "periodo": {},
  "saldos": {},
  "totales": {},
  "movimientos": [],
  "aplicaciones": [],
  "exclusiones": [],
  "pendientes": [],
  "usuario": {},
  "fecha": "...",
}
```

No necesariamente deben guardarse todos los objetos completos en JSON si existe `MovimientoPeriodoConciliacion`; el snapshot puede guardar totales, IDs y hashes.

11. Hash del cierre

Se recomienda calcular un hash sobre:

- Cuenta.
- Período.
- Movimientos.
- Aplicaciones.
- Exclusiones.
- Saldos.

```
hash_cierre = models.CharField(  
    max_length=64,  
    blank=True,  
)
```

Esto ayuda a detectar modificaciones posteriores al snapshot.

12. Efecto del cierre

Después de cerrar:

- No se agregan aplicaciones ordinarias.
- No se anulan aplicaciones.
- No se crean exclusiones.
- No se revocan exclusiones.
- No se modifica la cartola.
- No se modifica el saldo.
- No se cambia el período.

Las acciones requieren reapertura.

13. Servicio

```
services/conciliacion/cierre.py
```

```
@transaction.atomic  
def cerrar_conciliacion(  
    *,  
    conciliacion_id,  
    usuario,  
    motivo_diferencia="",  
):  
    conciliacion = (  
        ConciliacionBancaria.objects
```

```
        .select_for_update()
        .get(pk=conciliacion_id)
    )

    # Bloquear movimientos.
    # Recalcular totales.
    # Verificar pendientes.
    # Calcular diferencia.
    # Validar tolerancia.
    # Generar snapshot y hash.
    # Cambiar a CERRADA.
    # Registrar cierre.
```

Funciones:

```
crear_conciliacion_periodo()
calcular_resumen_periodo()
validar_cierre()
cerrar_conciliacion()
generar_snapshot()
calcular_hash_cierre()
bloquear_periodo()
```

14. URLs

```
/administracion/conciliaciones/
/administracion/conciliaciones/nueva/
/administracion/conciliaciones/<pk>/
/administracion/conciliaciones/<pk>/validar/
/administracion/conciliaciones/<pk>/cerrar/
/administracion/conciliaciones/<pk>/snapshot/
/administracion/conciliaciones/<pk>/certificado/
```

15. Vistas

```
ConciliacionBancariaListView
ConciliacionBancariaCreateView
ConciliacionBancariaDetailView
ConciliacionBancariaValidarView
ConciliacionBancariaCerrarView
ConciliacionBancariaSnapshotView
ConciliacionBancariaCertificadoView
```

16. Certificado de cierre

Debe contener:

- Cuenta enmascarada.
 - Banco.
 - Período.
 - Saldos.
 - Diferencia.
 - Totales.
 - Pendientes.
 - Exclusiones.
 - Usuario.
 - Fecha.
 - Número de revisión.
 - Hash.
-

17. Permisos

```
view_conciliacionbancaria  
add_conciliacionbancaria  
cerrar_conciliacionbancaria  
cerrar_conciliacion_con_diferencia  
ver_snapshot_conciliacion
```

18. Auditoría

Registrar:

- Creación.
 - Validación.
 - Cierre.
 - Diferencia.
 - Motivo.
 - Usuario.
 - Snapshot.
 - Hash.
 - Reapertura.
 - Nuevo cierre.
-

19. Pruebas

- Crear período.
- Rango inválido.

- Superposición.
 - Cierre sin pendientes.
 - Movimiento pendiente.
 - Movimiento parcial.
 - Excluido justificado.
 - Excluido sin motivo.
 - Importación con errores.
 - Diferencia cero.
 - Diferencia dentro de tolerancia.
 - Diferencia superior.
 - Usuario sin permiso.
 - Bloqueo posterior.
 - Snapshot.
 - Hash.
 - Reapertura.
 - Segundo cierre.
 - Concurrencia.
-

20. Criterios de aceptación

- Los períodos se crean por cuenta.
 - No existen superposiciones.
 - Los movimientos quedan congelados.
 - Los pendientes bloquean el cierre.
 - Las exclusiones justificadas son aceptadas.
 - La diferencia se calcula consistentemente.
 - Las diferencias autorizadas requieren motivo.
 - Se genera snapshot y hash.
 - El período cerrado queda bloqueado.
 - La reapertura crea una nueva revisión.
-

7. Estructura de archivos del bloque

```
administracion/  
├── models/  
│   ├── bancos.py  
│   └── conciliacion.py  
├── forms/  
│   ├── pagos_desde_movimiento.py  
│   ├── reglas_conciliacion.py  
│   ├── cruce_asistido.py  
│   ├── exclusiones.py  
│   ├── reversas.py  
│   └── cierre_conciliacion.py  
└──
```

```
├─ views/
│   ├── pagos_desde_movimiento.py
│   ├── reglas_conciliacion.py
│   ├── cruce_asistido.py
│   ├── aplicaciones_transversales.py
│   ├── exclusiones.py
│   ├── reversas.py
│   └── cierre_conciliacion.py
├─ services/conciliacion/
│   ├── saldos.py
│   ├── aplicaciones.py
│   ├── pagos_facturas.py
│   ├── reglas.py
│   ├── sugerencias.py
│   ├── asistente.py
│   ├── exclusiones.py
│   ├── reversas.py
│   └── cierre.py
├─ selectors/
│   ├── movimientos_conciliacion.py
│   ├── candidatos_conciliacion.py
│   ├── sugerencias_conciliacion.py
│   ├── periodos_conciliacion.py
│   └── resumen_conciliacion.py
├─ management/commands/
│   └── generar_sugerencias_conciliacion.py
├─ templates/administracion/conciliacion/
│   ├── pagos_facturas/
│   ├── reglas/
│   ├── sugerencias/
│   ├── asistida/
│   ├── exclusiones/
│   ├── reversas/
│   └── periodos/
├─ static/administracion/js/
│   ├── conciliacion_asistida.js
│   ├── pagos_facturas.js
│   └── cierre_conciliacion.js
└─ tests/conciliacion/
    ├── test_pagos_facturas.py
    ├── test_conciliacion_parcial.py
    ├── test_reglas.py
    ├── test_sugerencias.py
    └── test_cruce_asistido.py
```

```
|— test_cruce_transversal.py
|— test_exclusiones.py
|— test_reversas.py
|— test_cierre.py
|— test_concurrencia.py
```

8. Migraciones previstas

Migración 1 — Aplicaciones

Agregar:

```
anulada_en
anulada_por
motivo_anulacion
correlation_id
```

Agregar restricción entre destino y FK.

Migración 2 — Reglas

Crear:

```
ReglaCruceBancario
SugerenciaCruceBancario
```

Migración 3 — Exclusiones

Agregar:

```
tipo
activa
revocada_en
revocada_por
motivo_revocacion
```

Migración 4 — Conciliación

Ampliar:

```
saldos inicial y final
origen saldo libro
totales
contadores
snapshot
hash
datos de reapertura
número de revisión
```

Migración 5 — Movimientos del período

Crear, si se aprueba:

```
MovimientoPeriodoConciliacion
```

Migración 6 — Índices y restricciones

Agregar:

- Aplicaciones por movimiento y activa.
- Aplicaciones por destino.
- Reglas por cuenta, prioridad y activa.
- Sugerencias por movimiento y estado.
- Exclusión activa.
- Conciliación por cuenta y período.
- Control de superposición a nivel de servicio y, cuando el motor lo permita, base de datos.

9. Índices recomendados

AplicacionMovimientoBancario

```
models.Index(
    fields=["movimiento", "activa"],
)

models.Index(
    fields=["destino", "activa"],
)

models.Index(
```

```
        fields=["correlation_id"],
    )
```

ReglaCruceBancario

```
models.Index(
    fields=["cuenta_bancaria", "activa", "prioridad"],
)

models.Index(
    fields=["destino", "activa"],
)
```

SugerenciaCruceBancario

```
models.Index(
    fields=["movimiento", "estado"],
)

models.Index(
    fields=["estado", "-puntaje"],
)
```

ConciliacionBancaria

```
models.Index(
    fields=["cuenta_bancaria", "periodo_desde"],
)

models.Index(
    fields=["cuenta_bancaria", "periodo_hasta"],
)

models.Index(
    fields=["estado", "periodo_hasta"],
)
```

ExclusionMovimientoBancario

```
models.Index(
    fields=["activa", "tipo"],
)
```

10. Definición de terminado del bloque

B010–B017 se considerará terminado cuando:

- Un movimiento puede pagar varias facturas.
- La relación utiliza `PagoCliente`.
- Los movimientos admiten conciliación parcial.
- Los destinos admiten conciliación parcial.
- Se calculan saldos de ambos lados.
- Existen reglas configurables.
- Las reglas generan sugerencias explicables.
- El usuario puede realizar cruces asistidos.
- Se integran facturación, rendiciones y factoring.
- Las direcciones de movimientos se validan.
- Los movimientos pueden excluirse justificadamente.
- Las exclusiones no modifican la cartola.
- Las aplicaciones pueden reversarse.
- Las reversas conservan historial.
- Los períodos pueden reabrirse con permiso.
- Los períodos se cierran por cuenta.
- No existen períodos superpuestos.
- Los pendientes bloquean el cierre.
- Las diferencias requieren control.
- Se generan snapshots.
- Los períodos cerrados quedan bloqueados.
- Las operaciones son atómicas.
- Se controlan concurrencias.
- Las pruebas automáticas pasan.

11. Riesgos del bloque

Riesgo	Impacto	Tratamiento
Relacionar movimiento directamente con factura	Alto	Usar <code>PagoCliente</code>
Sobreaplicar movimiento	Alto	Bloqueo y validación
Sobreaplicar destino	Alto	Control bilateral
Confundir parcial con pendiente	Medio/alto	Estados y saldos explícitos
Regla automática incorrecta	Alto	Sugerencia por defecto
Sugerencia sin explicación	Medio	Guardar puntaje y razones
Doble aplicación automática	Alto	Idempotencia
Cruce manual concurrente	Alto	<code>select_for_update()</code>
Dirección bancaria incorrecta	Alto	Matriz de destinos

Riesgo	Impacto	Tratamiento
Exclusión usada para ocultar errores	Alto	Tipos, permisos y auditoría
Borrar aplicaciones	Alto	Anulación lógica
Reversa deja estados incoherentes	Alto	Propagación transaccional
Cerrar con pendientes	Alto	Validación previa
Períodos superpuestos	Alto	Restricción por servicio
Saldo libro sin origen	Alto	Campo de procedencia
Presentar saldo operativo como contable	Alto	Etiquetado explícito
Modificación posterior al cierre	Alto	Snapshot y hash
Reapertura sin evidencia	Alto	Revisiones y motivo
Diferencia autorizada sin control	Alto	Tolerancia y permiso especial

12. Resultado consolidado

Al finalizar este bloque existirá el ciclo:

```

Cartola importada
→ movimiento clasificado
→ candidatos sugeridos
→ cruce manual o automático
→ aplicación parcial o completa
→ integración con facturas, rendiciones y factoring
→ exclusiones justificadas
→ reversas controladas
→ validación del período
→ cierre con snapshot

```

La salida habilita:

```

B018 – Dashboard de conciliación
B019 – Alertas bancarias
B020 – Reportes y exportación
B021 – Pruebas de conciliación

```